Week 2 - In Class Activity: End-to-end Machine Learning Pipeline in R

Abigail Kreuser

Goal: Create an end-to-end machine learning regression pipeline in R using `tidymodels`.

In this activity, you will use an **abalone** dataset. The age of abalone is usually determined by cutting the shell through the cone, staining it, and counting the number of rings under a microscope. We will use easier-to-measure variables to predict the number of rings.

By the end of the activity, you should be able to:

1. Load and explore a dataset in R.
2. Separate predictors from the response variable.
3. Create new features from existing variables.
4. Build a reusable preprocessing recipe.
5. Fit and tune a machine learning model using cross-validation.
6. Evaluate model performance on held-out test data.
7. Interpret model behavior using predictions, residuals, and variable importance.
8. Modify the pipeline to use a different model.

R machine learning library: This R Markdown document uses `tidymodels`, the most common modern R ecosystem for machine learning workflows.

Dataset: Warwick J Nash, Tracy L Sellers, Simon R Talbot, Andrew J Cawthorn and Wes B Ford (1994) "The Population Biology of Abalone (*Haliotis* species) in Tasmania. I. Blacklip Abalone (*H. rubra*) from the North Coast and Islands of Bass Strait", Sea Fisheries Division, Technical Report No. 48 (ISSN 1034-3288).

1. Setup

This R Markdown document intentionally gives you some complete code and some incomplete code. Sections marked **TODO** are places where you should write or modify code yourself.

If you do not already have these packages installed, run the install cell once. Then you can comment it out or skip it later.

```
# Run this once if the packages are not installed.
#install.packages(c("tidymodels", "ranger", "kknn"))
```

```
library(tidymodels)
```

```
## -- Attaching packages ---------------------------------------------------------------------------
## v broom        1.0.13     v recipes      1.3.2
## v dials        1.4.4      v rsample      1.3.2
## v dplyr        1.2.1      v tailor       0.1.0
## v ggplot2      4.0.3      v tidyr        1.3.2
## v infer        1.1.0      v tune         2.1.0
## v modeldata    1.5.1      v workflows    1.3.0
## v parsnip      1.6.0      v workflowsets 1.1.1
## v purrr        1.2.2      v yardstick    1.4.0
```

```
## -- Conflicts ---------------------------------------------------------------------------------------
## x purrr::discard() masks scales::discard()
## x dplyr::filter()  masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## x recipes::step()  masks stats::step()
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ---------------------------------------------------------------
## v forcats   1.0.1     v stringr   1.6.0
## v lubridate 1.9.4     v tibble    3.3.0
## v readr     2.1.6
```

```
## -- Conflicts ---------------------------------------------------------------------------------------
## x readr::col_factor() masks scales::col_factor()
## x purrr::discard()    masks scales::discard()
## x dplyr::filter()     masks stats::filter()
## x stringr::fixed()    masks recipes::fixed()
## x dplyr::lag()        masks stats::lag()
## x readr::spec()       masks yardstick::spec()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
# Make results reproducible.
set.seed(42)

tidymodels_prefer()
```

2. Load the Data

The dataset does not include column names in the raw CSV, so we add them manually while reading the file.

```
column_names <- c(
  "sex", "length", "diameter", "height", "whole_weight",
  "shucked_weight", "viscera_weight", "shell_weight", "rings"
)

abalone <- read_csv(
  "https://raw.githubusercontent.com/dblaskey/ML_Course_Code/main/Data/abalone.csv",
  col_names = column_names,
  show_col_types = FALSE
)

abalone
```

```
## # A tibble: 4,177 x 9
##    sex   length diameter height whole_weight shucked_weight viscera_weight
##    <chr>  <dbl>    <dbl>  <dbl>        <dbl>          <dbl>          <dbl>
##  1 M      0.455    0.365  0.095        0.514         0.224          0.101
##  2 M      0.35     0.265  0.09         0.226         0.0995         0.0485
##  3 F      0.53     0.42   0.135        0.677         0.256          0.142
##  4 M      0.44     0.365  0.125        0.516         0.216          0.114
##  5 I      0.33     0.255  0.08         0.205         0.0895         0.0395
##  6 I      0.425    0.3    0.095        0.352         0.141          0.0775
##  7 F      0.53     0.415  0.15         0.778         0.237          0.142
##  8 F      0.545    0.425  0.125        0.768         0.294          0.150
##  9 M      0.475    0.37   0.125        0.509         0.216          0.112
## 10 F      0.55     0.44   0.15         0.894         0.314          0.151
```

```
## # i 4,167 more rows
## # i 2 more variables: shell_weight <dbl>, rings <dbl>
```

3. Data Exploration

Before fitting a model, we explore the dataset so we understand what each variable represents and what data types are present.

```
# Summary statistics for each variable.
summary(abalone)
```

```
##      sex                length         diameter          height
##  Length:4177        Min.   :0.075   Min.   :0.0550   Min.   :0.0000
##  Class :character   1st Qu.:0.450   1st Qu.:0.3500   1st Qu.:0.1150
##  Mode  :character   Median :0.545   Median :0.4250   Median :0.1400
##                     Mean   :0.524   Mean   :0.4079   Mean   :0.1395
##                     3rd Qu.:0.615   3rd Qu.:0.4800   3rd Qu.:0.1650
##                     Max.   :0.815   Max.   :0.6500   Max.   :1.1300
##   whole_weight    shucked_weight   viscera_weight    shell_weight
##  Min.   :0.0020   Min.   :0.0010   Min.   :0.0005   Min.   :0.0015
##  1st Qu.:0.4415   1st Qu.:0.1860   1st Qu.:0.0935   1st Qu.:0.1300
##  Median :0.7995   Median :0.3360   Median :0.1710   Median :0.2340
##  Mean   :0.8287   Mean   :0.3594   Mean   :0.1806   Mean   :0.2388
##  3rd Qu.:1.1530   3rd Qu.:0.5020   3rd Qu.:0.2530   3rd Qu.:0.3290
##  Max.   :2.8255   Max.   :1.4880   Max.   :0.7600   Max.   :1.0050
##      rings
##  Min.   : 1.000
##  1st Qu.: 8.000
##  Median : 9.000
##  Mean   : 9.934
##  3rd Qu.:11.000
##  Max.   :29.000
```

```
# Determine the number of missing values in each column.
abalone |>
  summarize(across(everything(), ~ sum(is.na(.x)))) |>
  pivot_longer(everything(), names_to = "variable", values_to = "missing_values") |>
  arrange(desc(missing_values))
```

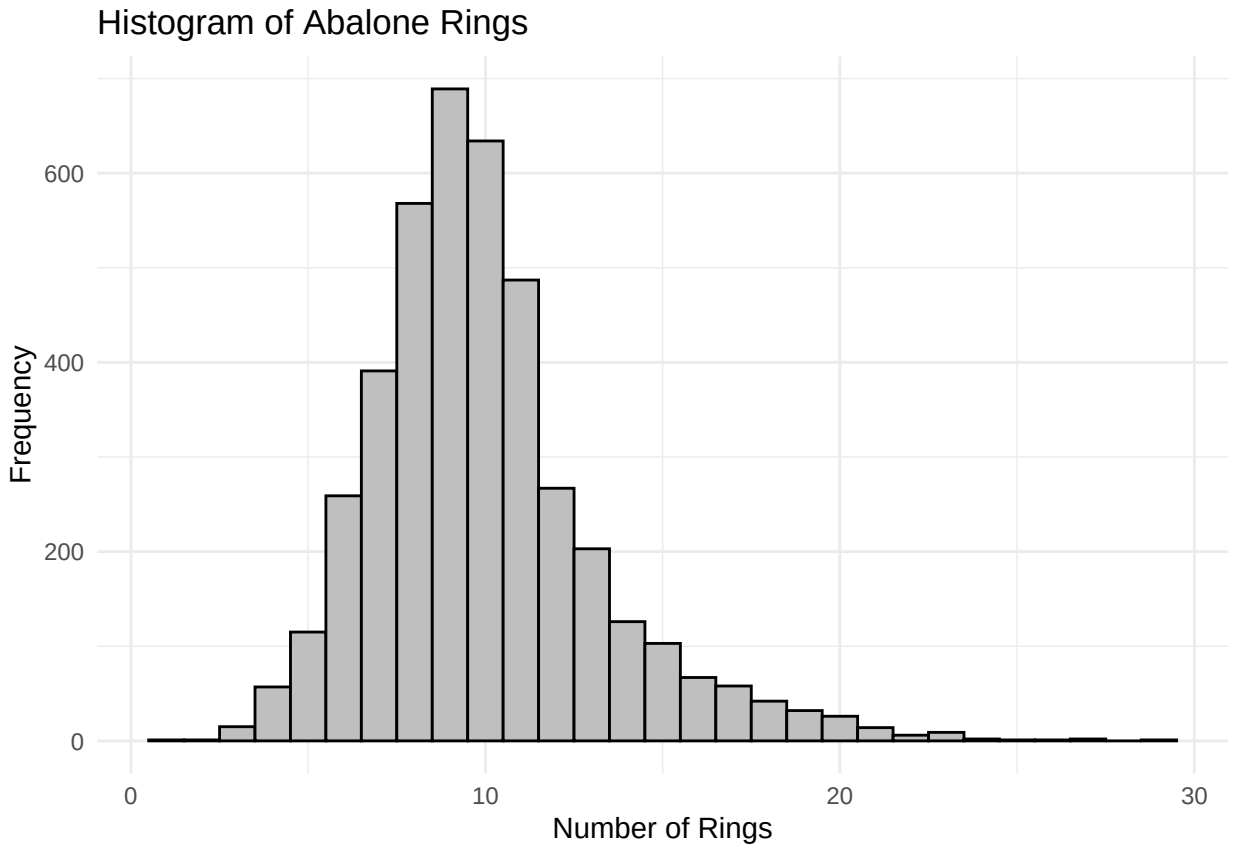
```
## # A tibble: 9 x 2
##   variable       missing_values
##   <chr>                   <int>
## 1 sex                         0
## 2 length                      0
## 3 diameter                    0
## 4 height                      0
## 5 whole_weight                0
## 6 shucked_weight              0
## 7 viscera_weight              0
## 8 shell_weight                0
## 9 rings                       0
```

```
# Determine the column types.
glimpse(abalone)
```

```
## Rows: 4,177
## Columns: 9
```

```
## $ sex            <chr> "M", "M", "F", "M", "I", "I", "F", "F", "M", "F", "F", ~
## $ length         <dbl> 0.455, 0.350, 0.530, 0.440, 0.330, 0.425, 0.530, 0.545,~
## $ diameter       <dbl> 0.365, 0.265, 0.420, 0.365, 0.255, 0.300, 0.415, 0.425,~
## $ height         <dbl> 0.095, 0.090, 0.135, 0.125, 0.080, 0.095, 0.150, 0.125,~
## $ whole_weight   <dbl> 0.5140, 0.2255, 0.6770, 0.5160, 0.2050, 0.3515, 0.7775,~
## $ shucked_weight <dbl> 0.2245, 0.0995, 0.2565, 0.2155, 0.0895, 0.1410, 0.2370,~
## $ viscera_weight <dbl> 0.1010, 0.0485, 0.1415, 0.1140, 0.0395, 0.0775, 0.1415,~
## $ shell_weight   <dbl> 0.150, 0.070, 0.210, 0.155, 0.055, 0.120, 0.330, 0.260,~
## $ rings          <dbl> 15, 7, 9, 10, 7, 8, 20, 16, 9, 19, 14, 10, 11, 10, 10, ~
```

```r
ggplot(abalone, aes(x = rings)) +
  geom_histogram(binwidth = 1, color = "black", fill = "gray75") +
  labs(
    x = "Number of Rings",
    y = "Frequency",
    title = "Histogram of Abalone Rings"
  ) +
  theme_minimal()
```



4. Model Preprocessing and Regression

We now build a model for the number of abalone rings.

The response variable is `rings`.

The predictors are:

Name	Data Type	Units	Description
sex	nominal		M, F, and I infant
length	continuous	mm	Longest shell measurement
diameter	continuous	mm	Perpendicular to length
height	continuous	mm	With meat in shell
whole_weight	continuous	grams	Whole abalone
shucked_weight	continuous	grams	Weight of meat
viscera_weight	continuous	grams	Gut weight after bleeding
shell_weight	continuous	grams	After being dried

Train/Test Split

We split the data into training and testing sets. Because `rings` is numeric, we use `strata = rings` so the split tries to keep similar distributions of abalone ages in both sets.

```
set.seed(42)

abalone_split <- initial_split(
  abalone,
  prop = 0.80,
  strata = rings
)

abalone_train <- training(abalone_split)
abalone_test  <- testing(abalone_split)

abalone_train
```

```
## # A tibble: 3,340 x 9
##    sex   length diameter height whole_weight shucked_weight viscera_weight
##    <chr>  <dbl>    <dbl>  <dbl>        <dbl>          <dbl>          <dbl>
##  1 M      0.35     0.265  0.09         0.226         0.0995         0.0485
##  2 I      0.33     0.255  0.08         0.205         0.0895         0.0395
##  3 I      0.425    0.3    0.095        0.352         0.141          0.0775
##  4 M      0.465    0.355  0.105        0.480         0.227          0.124
##  5 F      0.45     0.355  0.105        0.522         0.237          0.116
##  6 I      0.24     0.175  0.045        0.07          0.0315         0.0235
##  7 I      0.21     0.15   0.05         0.042         0.0175         0.0125
##  8 I      0.39     0.295  0.095        0.203         0.0875         0.045
##  9 F      0.46     0.375  0.12         0.460         0.178          0.11
## 10 I      0.325    0.245  0.07         0.161         0.0755         0.0255
## # i 3,330 more rows
## # i 2 more variables: shell_weight <dbl>, rings <dbl>
```

```
abalone_test
```

```
## # A tibble: 837 x 9
##    sex   length diameter height whole_weight shucked_weight viscera_weight
##    <chr>  <dbl>    <dbl>  <dbl>        <dbl>          <dbl>          <dbl>
##  1 M      0.455    0.365  0.095        0.514          0.224          0.101
##  2 F      0.53     0.42   0.135        0.677          0.256          0.142
##  3 F      0.55     0.44   0.15         0.894          0.314          0.151
##  4 F      0.525    0.38   0.14         0.606          0.194          0.148
##  5 I      0.355    0.28   0.085        0.290          0.095          0.0395
##  6 F      0.44     0.34   0.1          0.451          0.188          0.087
```

```
##  7 M      0.365    0.295  0.08         0.256          0.097         0.043
##  8 M      0.59     0.445  0.14         0.931          0.356         0.234
##  9 M      0.575    0.425  0.14         0.864          0.393         0.227
## 10 F      0.575    0.445  0.135        0.883          0.381         0.204
## # i 827 more rows
## # i 2 more variables: shell_weight <dbl>, rings <dbl>
```

Feature Engineering

Feature engineering creates new predictors from existing predictors. We will talk more about this next week.

In `tidymodels`, feature engineering usually belongs inside a `recipe`. This is important because the same transformations are applied consistently during training, cross-validation, and prediction.

```
abalone_recipe <- recipe(rings ~ ., data = abalone_train) |>
  # Shape proxy
  step_mutate(volume_proxy = length * diameter * height) |>
  # Composition ratios. The small constant prevents division by zero.
  step_mutate(shell_to_whole = shell_weight / (whole_weight + 1e-6)) |>
  step_mutate(weight_per_size = whole_weight / (volume_proxy + 1e-6)) |>
  # Residual-weight feature
  step_mutate(nonmeat_weight = whole_weight - shucked_weight - viscera_weight) |>
  # Preprocessing
  step_impute_median(all_numeric_predictors()) |>
  step_impute_mode(all_nominal_predictors()) |>
  step_dummy(all_nominal_predictors(), one_hot = FALSE) |>
  step_normalize(all_numeric_predictors())

abalone_recipe
```

```
##
## -- Recipe ----------------------------------------------------------------------------------------
##
## -- Inputs
## Number of variables by role
## outcome:   1
## predictor: 8
##
## -- Operations
## * Variable mutation for: length * diameter * height
## * Variable mutation for: shell_weight / (whole_weight + 1e-06)
## * Variable mutation for: whole_weight / (volume_proxy + 1e-06)
## * Variable mutation for: whole_weight - shucked_weight - viscera_weight
## * Median imputation for: all_numeric_predictors()
## * Mode imputation for: all_nominal_predictors()
## * Dummy variables from: all_nominal_predictors()
## * Centering and scaling for: all_numeric_predictors()
```

Build the Random Forest Workflow

We are using a **random forest** model, but do not worry too much about the details of this model or its hyperparameters yet. We will discuss random forests later in the semester. Right now, we are using it to demonstrate the end-to-end machine learning pipeline.

The workflow combines:

1. A preprocessing recipe.
2. A model specification.
3. A tuning grid.
4. Cross-validation.

```
rf_model <- rand_forest(
  trees = tune(),
  mtry = tune(),
  min_n = tune()
) |>
  set_engine("ranger", importance = "permutation") |>
  set_mode("regression")

rf_workflow <- workflow() |>
  add_recipe(abalone_recipe) |>
  add_model(rf_model)

rf_workflow
```

```
## == Workflow ====================================================================================
## Preprocessor: Recipe
## Model: rand_forest()
##
## -- Preprocessor ----------------------------------------------------------------------------
## 8 Recipe Steps
##
## * step_mutate()
## * step_mutate()
## * step_mutate()
## * step_mutate()
## * step_impute_median()
## * step_impute_mode()
## * step_dummy()
## * step_normalize()
##
## -- Model -----------------------------------------------------------------------------------
## Random Forest Model Specification (regression)
##
## Main Arguments:
##   mtry = tune()
##   trees = tune()
##   min_n = tune()
##
## Engine-Specific Arguments:
##   importance = permutation
##
## Computational engine: ranger
```

Cross-Validation

Cross-validation gives a more realistic estimate of predictive performance than judging the model only on the training data. The training dataset is split into five folds. The model is trained on four folds and validated on the remaining fold, repeated five times.

In this example, we tune the random forest using **RMSE**. Lower RMSE means better predictive performance.

```
set.seed(42)

abalone_folds <- vfold_cv(
  abalone_train,
  v = 5,
  strata = rings
)

rf_grid <- grid_regular(
  trees(range = c(300, 500)),
  mtry(range = c(2, 6)),
  min_n(range = c(4, 8)),
  levels = c(trees = 2, mtry = 3, min_n = 3)
)

rf_grid
```

```
## # A tibble: 18 x 3
##    trees  mtry min_n
##    <int> <int> <int>
##  1   300     2     4
##  2   500     2     4
##  3   300     4     4
##  4   500     4     4
##  5   300     6     4
##  6   500     6     4
##  7   300     2     6
##  8   500     2     6
##  9   300     4     6
## 10   500     4     6
## 11   300     6     6
## 12   500     6     6
## 13   300     2     8
## 14   500     2     8
## 15   300     4     8
## 16   500     4     8
## 17   300     6     8
## 18   500     6     8
```

```
# WARNING: This cell may take a few minutes to run.

set.seed(42)

rf_tuned <- tune_grid(
  rf_workflow,
  resamples = abalone_folds,
  grid = rf_grid,
  metrics = metric_set(rmse, mae, rsq),
```

```
  control = control_grid(save_pred = TRUE)
)

collect_metrics(rf_tuned) |>
  arrange(mean)
```

```
## # A tibble: 54 x 9
##     mtry trees min_n .metric .estimator  mean     n std_err .config
##    <int> <int> <int> <chr>   <chr>      <dbl> <int>   <dbl> <chr>
##  1     6   300     4 rsq     standard   0.558     5  0.0172 pre0_mod13_post0
##  2     6   300     6 rsq     standard   0.558     5  0.0162 pre0_mod14_post0
##  3     2   500     4 rsq     standard   0.559     5  0.0164 pre0_mod04_post0
##  4     4   300     4 rsq     standard   0.560     5  0.0170 pre0_mod07_post0
##  5     2   300     4 rsq     standard   0.560     5  0.0168 pre0_mod01_post0
##  6     6   300     8 rsq     standard   0.560     5  0.0171 pre0_mod15_post0
##  7     6   500     6 rsq     standard   0.560     5  0.0167 pre0_mod17_post0
##  8     6   500     4 rsq     standard   0.560     5  0.0167 pre0_mod16_post0
##  9     6   500     8 rsq     standard   0.560     5  0.0169 pre0_mod18_post0
## 10     2   300     8 rsq     standard   0.561     5  0.0172 pre0_mod03_post0
## # i 44 more rows
```

```
best_rf <- select_best(rf_tuned, metric = "rmse")
best_rf #   4   300     6 pre0_mod08_post0
```

```
## # A tibble: 1 x 4
##    mtry trees min_n .config
##   <int> <int> <int> <chr>
## 1     4   300     6 pre0_mod08_post0
```

```
final_rf_workflow <- finalize_workflow(rf_workflow, best_rf)

final_rf_fit <- last_fit(
  final_rf_workflow,
  split = abalone_split,
  metrics = metric_set(rmse, mae, rsq)
)

collect_metrics(final_rf_fit)
```

```
## # A tibble: 3 x 4
##   .metric .estimator .estimate .config
##   <chr>   <chr>          <dbl> <chr>
## 1 rmse    standard       2.16  pre0_mod0_post0
## 2 mae     standard       1.50  pre0_mod0_post0
## 3 rsq     standard       0.578 pre0_mod0_post0
```

Evaluate Predictions

Now we collect predictions for the held-out test set and visualize how close the model predictions were to the observed number of rings.

```
rf_predictions <- collect_predictions(final_rf_fit) |>
  mutate(residual = rings - .pred)

rf_predictions |>
  select(rings, .pred, residual) |>
```

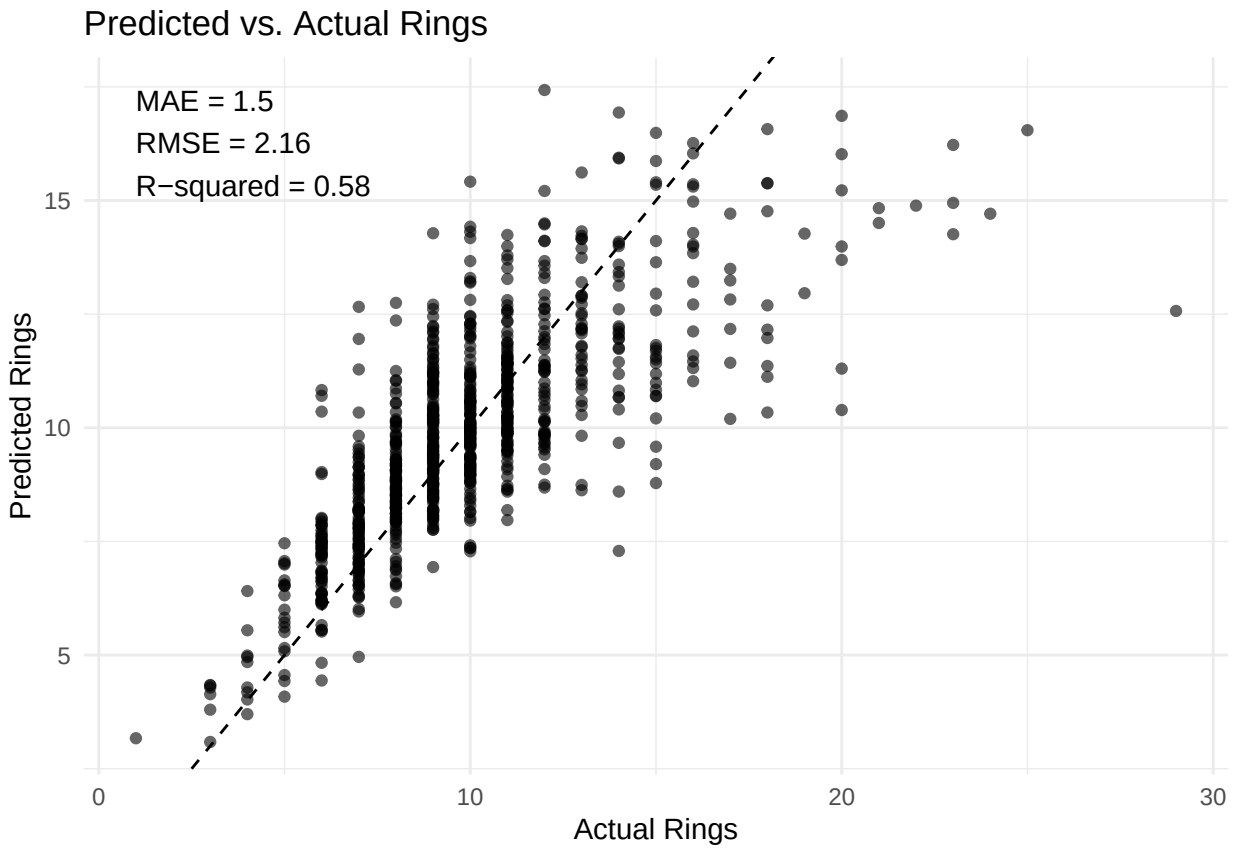
```
  head()
```

```
## # A tibble: 6 x 3
##   rings .pred residual
##   <dbl> <dbl>    <dbl>
## 1    15  9.20    5.80
## 2     9 10.8    -1.83
## 3    19 14.3     4.73
## 4    14 11.4     2.55
## 5     7  8.95   -1.95
## 6    10  9.17    0.826
```

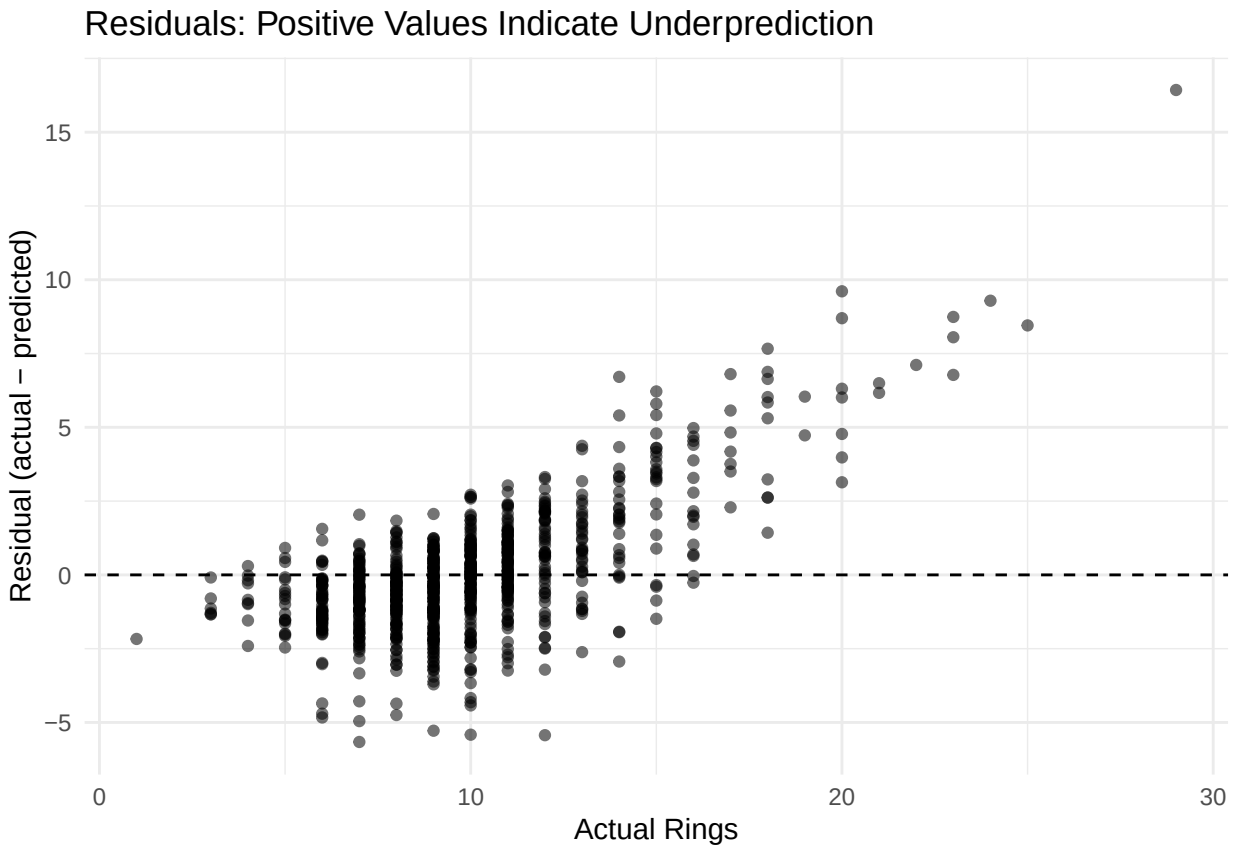
```
rf_metrics <- collect_metrics(final_rf_fit)

rmse_value <- rf_metrics |> filter(.metric == "rmse") |> pull(.estimate)
mae_value  <- rf_metrics |> filter(.metric == "mae")  |> pull(.estimate)
rsq_value  <- rf_metrics |> filter(.metric == "rsq")  |> pull(.estimate)

ggplot(rf_predictions, aes(x = rings, y = .pred)) +
  geom_point(alpha = 0.6) +
  geom_abline(linetype = "dashed") +
  annotate(
    "text",
    x = min(rf_predictions$rings),
    y = max(rf_predictions$.pred),
    hjust = 0,
    vjust = 1,
    label = paste0(
      "MAE = ", round(mae_value, 2),
      "\nRMSE = ", round(rmse_value, 2),
      "\nR-squared = ", round(rsq_value, 2)
    )
  ) +
  labs(
    x = "Actual Rings",
    y = "Predicted Rings",
    title = "Predicted vs. Actual Rings"
  ) +
  theme_minimal()
```




```
ggplot(rf_predictions, aes(x = rings, y = residual)) +
  geom_point(alpha = 0.55) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  labs(
    x = "Actual Rings",
    y = "Residual (actual - predicted)",
    title = "Residuals: Positive Values Indicate Underprediction"
  ) +
  theme_minimal()
```



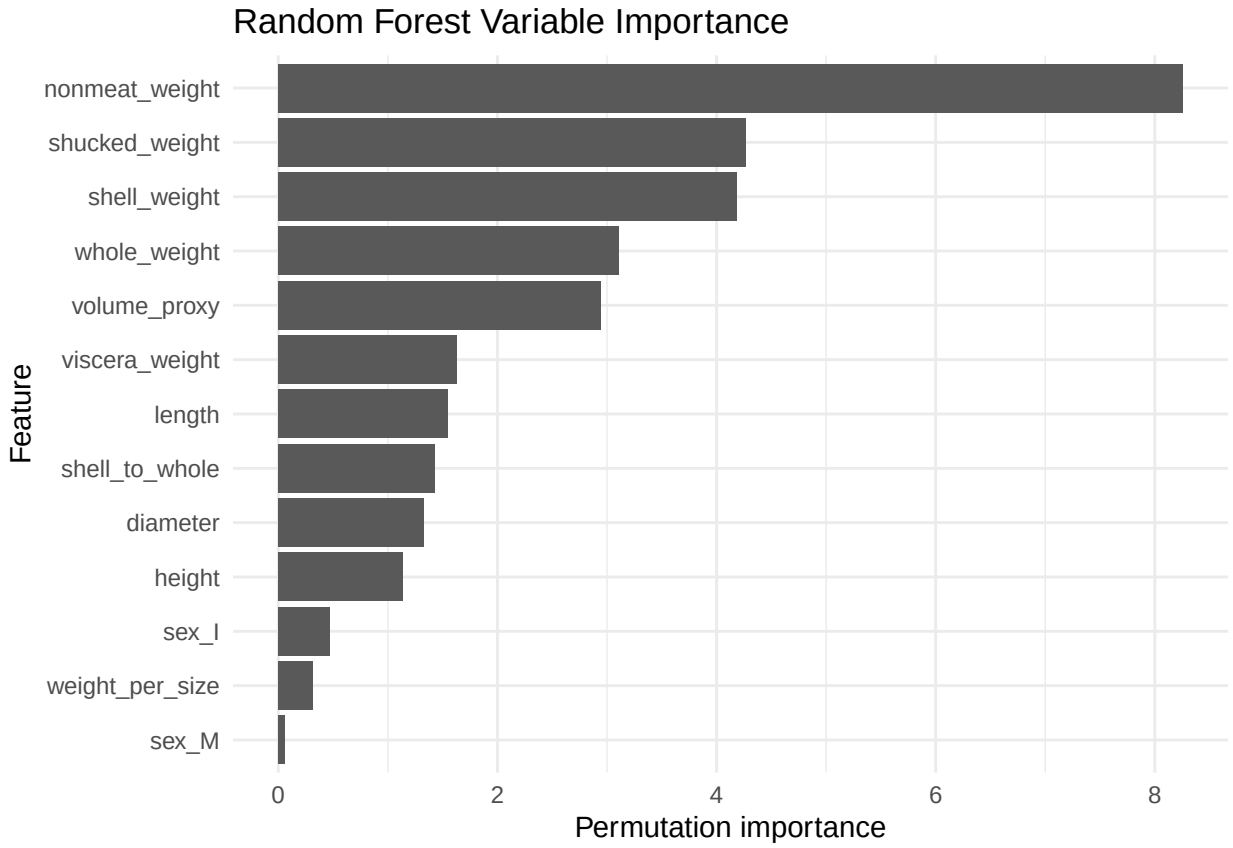
Variable Importance

Variable importance indicates which predictors provided the model with useful information during training. We will discuss this in more detail later in the semester.

```
final_rf_model <- extract_workflow(final_rf_fit) |>
  extract_fit_parsnip()

importance <- final_rf_model$fit$variable.importance |>
  enframe(name = "feature", value = "importance") |>
  arrange(desc(importance)) |>
  slice_head(n = 15) |>
  mutate(feature = reorder(feature, importance))

ggplot(importance, aes(x = feature, y = importance)) +
  geom_col() +
  coord_flip() +
  labs(
    x = "Feature",
    y = "Permutation importance",
    title = "Random Forest Variable Importance"
  ) +
  theme_minimal()
```



5. Reflection Questions

Answer these in a few sentences each.

1. What are the required steps in this machine learning pipeline? Split the data by stratified groups of the numeric response variable (abalone rings) to keep the testing and trainging groups similar distributions. Engineer features to make new predictor variables with imputationof a median, mode, and dummy variable then all are normalized which I assume fit better in the recipe. Then build the model workflow by incorporating a preprocessing recipe, model specifications, a tuning grid, and cross validation.
2. What train/test split was used for this model? An 80/20 percent split stratified following the distribution of ring data.
3. What does cross-validation do, and why do we use it? It seperates the training data further, reserving a fold for testing folds of training data which gives a better understanding of model performance than just one validation of testing/training data.
4. Which metric did we use to choose the best model? RMSE: root mean squared error
5. Did the model perform well? What evidence supports your answer? Using RMSE, I would say yes, because it is on the same scale as the target variable and 2 rings off doesn't seem too bad if I am only considering the numbers. However, for harvesting and management, I would need to know more to understand the reproductive value with age of abalone, and determine if being off two years in age estimation at certain ages would have population implications.
6. Look at the residual plot. Does the model make larger errors for younger or older abalone? The model makes larger errors for older abalone with large ranges after about 12 rings.
7. Which predictors appear most important? Do those results make sense biologically? All of the weight variables appear the most important, non-meat weight having the most importance which was calculated

by subtracting the shucked weight and viscera (gut weight after bleeding) from the whole weight. I think the results somewhat make sense biologically, but I don't understand why non-meat weight would be much more important than the dry shell weight. With the subtraction of the shucked weight adn viscera, the non-meat weight would be shell weight and anything wet that remained in the abalone after shucking, plus blood from the gut. Shell weight makes sense as being influential because the rings are made as the shell grows, however I would think the viscera would be accounted for in shucked weight and the weight of blood would be related to viscera size and meat.

6. Change the Pipeline

TODO: Update the pipeline to run a **K-nearest neighbors regression** model instead of a random forest.

Use the same training/test split, recipe, cross-validation folds, and evaluation metrics.

Hints: tidymodels:: - The `tidymodels` model function is `nearest_neighbor()`. - Use `set_engine("kknn")`. - Use `set_mode("regression")`. - Tune at least `neighbors`. - You may also tune `weight_func` and `dist_power`. - You may need to install the `kknn` package with `install.packages("kknn")`.

```
# TODO: Build a KNN model specification.
knn_model <- nearest_neighbor(
  neighbors = tune(),
  weight_func = tune(), #- You may also tune `weight_func` and `dist_power`.
   dist_power = tune()
) |>
  set_engine("kknn") |>
  set_mode("regression") |>
  translate()

# TODO: Combine the existing abalone_recipe with your KNN model in a workflow.
knn_workflow <- workflow() |>
  add_recipe(abalone_recipe) |>
  add_model(knn_model)

knn_workflow
```

```
## == Workflow ====================================================================================
## Preprocessor: Recipe
## Model: nearest_neighbor()
##
## -- Preprocessor ----------------------------------------------------------------------------
## 8 Recipe Steps
##
## * step_mutate()
## * step_mutate()
## * step_mutate()
## * step_mutate()
## * step_impute_median()
## * step_impute_mode()
## * step_dummy()
## * step_normalize()
##
## -- Model -----------------------------------------------------------------------------------
## K-Nearest Neighbor Model Specification (regression)
##
## Main Arguments:
##   neighbors = tune()
```

```
##   weight_func = tune()
##   dist_power = tune()
##
## Computational engine: kknn
##
## Model fit template:
## kknn::train.kknn(formula = missing_arg(), data = missing_arg(),
##     ks = min_rows(tune(), data, 5), kernel = tune(), distance = tune())
```

```
# TODO: Create a tuning grid for KNN.
set.seed(42)

abalone_folds <- vfold_cv(
  abalone_train,
  v = 5,
  strata = rings
)

knn_grid <- grid_regular(
  neighbors(range = c(1, 50)),
  weight_func(values = c("rectangular", "triangular", "epanechnikov", "gaussian")),
  dist_power(range = c(1, 2)),
  levels = c(neighbors = 10, weight_func = 4, dist_power = 2)
)

knn_grid
```

```
## # A tibble: 80 x 3
##    neighbors weight_func dist_power
##        <int> <chr>            <dbl>
##  1         1 rectangular          1
##  2         6 rectangular          1
##  3        11 rectangular          1
##  4        17 rectangular          1
##  5        22 rectangular          1
##  6        28 rectangular          1
##  7        33 rectangular          1
##  8        39 rectangular          1
##  9        44 rectangular          1
## 10        50 rectangular          1
## # i 70 more rows
```

```
# KNN model specification
knn_spec <- nearest_neighbor(
  neighbors   = tune(),
  weight_func = tune(),
  dist_power  = tune()
) %>%
  set_mode("classification") %>%   # use "regression" for numeric outcomes
  set_engine("kknn")



# Tune across the grid using cross-validation
```

```
# TODO: Tune the KNN model with cross-validation.
knn_tuned <- tune_grid(
  knn_workflow,
  resamples = abalone_folds,
  grid = knn_grid,
    metrics = metric_set(rmse, mae, rsq),
  control = control_grid(save_pred = TRUE)
  #metrics = metric_set(accuracy, roc_auc)  # choose metrics for your outcome
)

collect_metrics(knn_tuned) |>
  arrange(mean)
```

```
## # A tibble: 240 x 9
##    neighbors weight_func  dist_power .metric .estimator  mean     n std_err
##        <int> <chr>             <dbl> <chr>   <chr>      <dbl> <int>   <dbl>
##  1         1 epanechnikov          1 rsq     standard   0.307     5  0.0242
##  2         1 gaussian              1 rsq     standard   0.307     5  0.0242
##  3         1 rectangular           1 rsq     standard   0.307     5  0.0242
##  4         1 triangular            1 rsq     standard   0.307     5  0.0242
##  5         1 epanechnikov          2 rsq     standard   0.313     5  0.0256
##  6         1 rectangular           2 rsq     standard   0.313     5  0.0256
##  7         1 triangular            2 rsq     standard   0.313     5  0.0256
##  8         1 gaussian              2 rsq     standard   0.313     5  0.0256
##  9         6 triangular            1 rsq     standard   0.470     5  0.0208
## 10         6 epanechnikov          1 rsq     standard   0.475     5  0.0209
## # i 230 more rows
## # i 1 more variable: .config <chr>
```

```
# TODO: Select the best KNN model, fit it to the training set, and evaluate it on the test set.
best_knn <- select_best(knn_tuned, metric = "rmse")
best_knn #1        39 triangular           1 pre0_mod63_post0
```

```
## # A tibble: 1 x 4
##   neighbors weight_func dist_power .config
##       <int> <chr>            <dbl> <chr>
## 1        39 triangular           1 pre0_mod63_post0
```

```
final_knn_workflow <- finalize_workflow(knn_workflow, best_knn)

final_knn_fit <- last_fit(
  final_knn_workflow ,
  split = abalone_split,
  metrics = metric_set(rmse, mae, rsq)
)
collect_metrics(final_knn_fit)
```

```
## # A tibble: 3 x 4
##   .metric .estimator .estimate .config
##   <chr>   <chr>          <dbl> <chr>
## 1 rmse    standard       2.23  pre0_mod0_post0
## 2 mae     standard       1.49  pre0_mod0_post0
## 3 rsq     standard       0.569 pre0_mod0_post0
```

```
final_knn_model <- extract_workflow(final_knn_fit) |>
  extract_fit_parsnip()
```

```r
knn_predictions <- collect_predictions(final_knn_fit) |>
  mutate(residual = rings - .pred)

knn_predictions |>
  select(rings, .pred, residual) |>
  head()
```

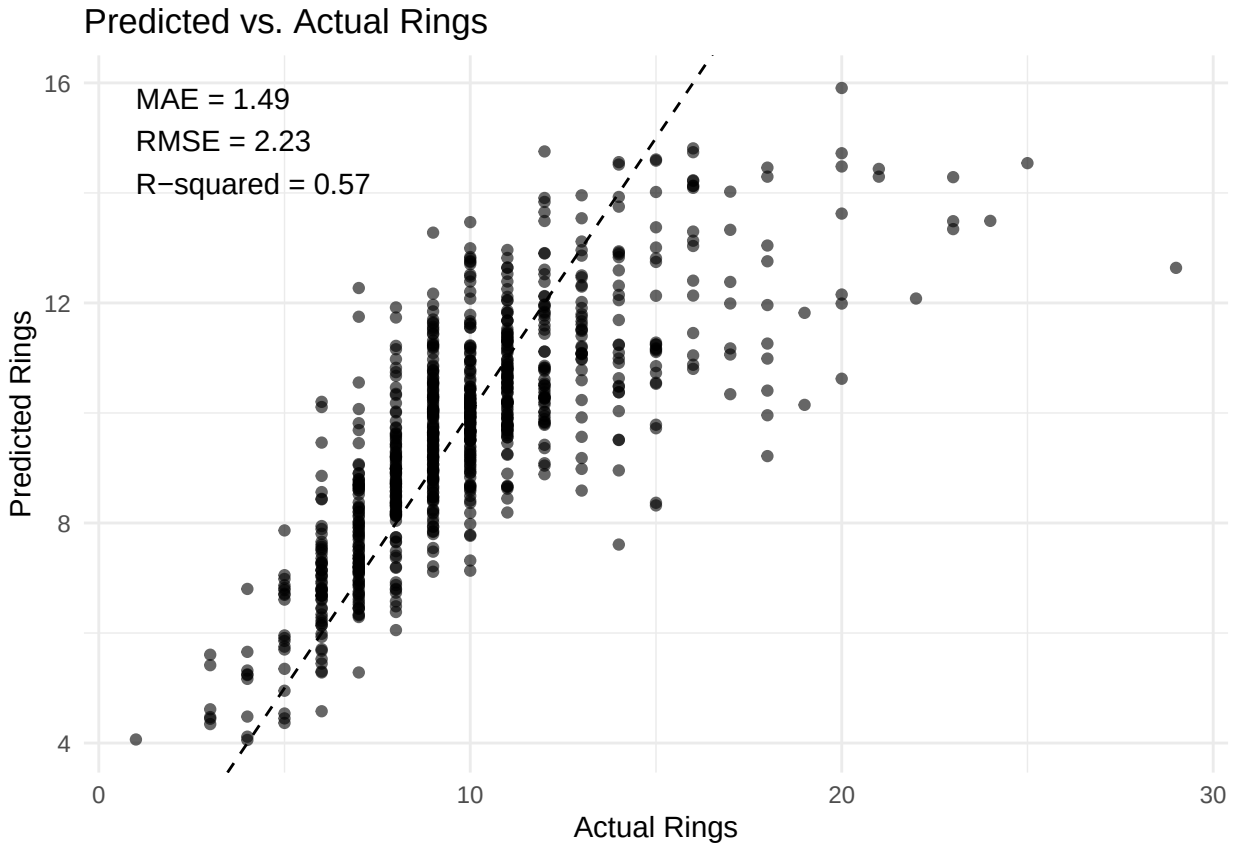
```
## # A tibble: 6 x 3
##   rings .pred residual
##   <dbl> <dbl>    <dbl>
## 1    15  8.37    6.63
## 2     9 11.2    -2.23
## 3    19 11.8     7.18
## 4    14 12.1     1.85
## 5     7  7.49   -0.492
## 6    10  9.37    0.630
```

```r
knn_metrics <- collect_metrics(final_knn_fit)

rmse_value <- knn_metrics |> filter(.metric == "rmse") |> pull(.estimate)
mae_value  <- knn_metrics |> filter(.metric == "mae")  |> pull(.estimate)
rsq_value  <- knn_metrics |> filter(.metric == "rsq")  |> pull(.estimate)

# super simmilar output...

ggplot(knn_predictions, aes(x = rings, y = .pred)) +
  geom_point(alpha = 0.6) +
  geom_abline(linetype = "dashed") +
  annotate(
    "text",
    x = min(knn_predictions$rings),
    y = max(knn_predictions$.pred),
    hjust = 0,
    vjust = 1,
    label = paste0(
      "MAE = ", round(mae_value, 2),
      "\nRMSE = ", round(rmse_value, 2),
      "\nR-squared = ", round(rsq_value, 2)
    )
  ) +
  labs(
    x = "Actual Rings",
    y = "Predicted Rings",
    title = "Predicted vs. Actual Rings"
  ) +
  theme_minimal()
```


Predicted vs. Actual Rings
MAE = 1.49
RMSE = 2.23
R−squared = 0.57
Predicted Rings
Actual Rings
16
12
8
4
0
10
20
30